

Análise e Desenho de Algoritmos 2025/26 – Trabalho 2

Sérgio Garrido (Nº 67202) e Francisco Oliveira (Nº 67711)

Faculdade de Ciências e Tecnologia - Universidade NOVA de Lisboa

1 Resolução do Problema

Neste projeto, a realização do controlo de interações de *beams* uns com os outros, foi o ponto principal de debate dentro do grupo. A seguinte explicação irá abordar a lógica utilizada, de modo a obter a solução desejada do problema proposto.

Visto que cada *beam* é composto por várias informações, nomeadamente em que linha e coluna têm origem, bem como o seu comprimento e direção, optou-se pela criação de uma classe *Beam* especializada, responsável por registar a informação de cada *beam*.

Seguidamente, sabendo que cada *beam* ocupará um determinado espaço no *grid* do problema, optou-se por criar as classes auxiliares *BeamBoundary* e *BeamCoordinates* responsáveis por guardar a informação relativa aos dados imutáveis dos objetos em questão. Assim, é possível pensar em cada *beam* como um retângulo no espaço do problema. Após a leitura das informações de entrada, todo o conteúdo é processado na classe *Solver*, onde a lógica para a solução do problema será aplicada.

A resolução do problema inicia-se com a determinação dos limites de cada *beam*. Como cada *beam* poderá apresentar uma direção distinta, será necessário realizar cálculos genéricos que, de acordo com a direção, permitem obter a informação correta relativamente às posições dos limites deste *beam* no *grid*. Por exemplo, se um *beam* se desloca para norte, a informação de entrada recebida pela leitura do problema indica a sua linha e coluna originais, bem como a sua direção e comprimento. No exemplo em questão, a definição de limites determinará que a linha de origem será a linha máxima a ser ocupada no *grid* pelo *beam*. Ao adicionar o seu comprimento desde esta linha de origem em direção norte, isto é, diminuindo o número de linhas, obteremos a linha mínima ocupada no *grid* pelo *beam* em questão. É de notar que como a direção é norte, os limites de coluna mínima e máximos ocupados serão idênticos, isto é, a coluna original. A mesma lógica poderá rapidamente e facilmente aplicada a todas as outras direções.

Após calcular os limites de cada *beam*, procede-se ao preenchimento do grafo do problema. Este grafo tem como objetivo definir um vetor de lista de adjacências de sucessores. Na solução apresentada, um sucessor é qualquer *beam* que necessite que o *beam* atual seja removido antes de si mesmo. Considere dois *beams*, A e B, se A se move em direção Este e B se move em direção Norte, de tal maneira que A ao ser libertado primeiro irá entrar em conflito com B, então B terá de ser removido antes de A. Esta relação será expressa no grafo tal que “A é sucessor de B”. Ainda que a lógica possa parecer invertida, este modo de pensamento permite facilmente ajustar o algoritmo de ordem topológica de Kahn ao problema em questão.

Ainda que a lógica por detrás do preenchimento do grafo do problema já esteja bem definida, é necessário entender como é que é decidido se um *beam* intersecta outro. Como a informação relativamente aos limites de cada *beam* já foi determinada e calculada, rapidamente se realiza o preenchimento de uma matriz de R linhas por C colunas, onde em cada índice, ou quadrado da matriz será colocado o número identificador (id) relativo ao *beam* que “habita” nesse espaço. Posteriormente, para cada *beam* e de acordo com a sua direção, a matriz será percorrida e à medida que se for descobrindo *beams* que serão intersectados aquando da libertação do *beam* atual, será realizada a adição destas relações no grafo de adjacências, bem como num vetor de inteiros, correspondente ao número de *beams* que intersectam o *beam* atual. Assim, após toda esta informação estar corretamente determinada, será possível prosseguir à aplicação dos vários algoritmos que solucionaram o problema.

Como o espaço do problema permite a inserção de aproximadamente 10000 *beams*, é necessário reduzir o espaço de exploração do algoritmo de ordenação topológica. Felizmente, como a informação do problema inclui as colunas que são necessárias libertar, podemos rapidamente determinar qual o conjunto de *beams* que

habita nestas colunas, pois estes serão obrigatoriamente parte do resultado a apresentar. Assim, utiliza-se um vetor de booleanos, que nos permite para cada *beam* id, saber se é ou não necessário de remover para chegar a um resultado. É de notar que ainda que já se saiba à partida os *beams* que têm de ser removidos, é necessário analisar se outros *beams* não bloqueiam um dos necessários. Como tal, sempre que se encontrar um *beam* que não seja necessário, percorremos os seus sucessores e, se porventura, se encontrar um sucessor que é necessário para a resolução do problema, então o *beam* inicial (*root*) será também, por transitividade, necessário. Repete-se este processo até não haver alterações no vetor de booleanos que representa a necessidade de remoção de *beams*. Caso o número de *beams* necessários seja zero, então estamos perante um “*False alarm*”, caso contrário será realizada a ordenação topológica.

Na solução apresentada optou-se por realizar a ordenação topológica e a determinação de ciclos de uma vez só. Assim, utiliza-se uma fila de prioridade mínima para rapidamente ordenar os *beams* de acordo com o seu id, sendo esta fila inicializada com todos os *beams* identificados como necessários no passo anterior, desde que, o número de *beams* que intersecta este *beam* necessário seja zero. Esta última condição é validada pelo vetor de inteiros *inDegree*, e é necessária para garantir que um *beam* pode ser removido sem intersectar outro aquando da sua libertação. Ao retirar um *beam* da fila, sabemos que este fará parte do resultado do problema pelo que será automaticamente adicionado ao resultado. Seguidamente, obtemos os seus sucessores, diminuindo o número de *beams* que os intersectam (informação presente no vetor de inteiros *inDegree*) em uma unidade, adicionando à fila aqueles que cumprirem as condições previamente enunciadas. O processo é repetido até que a fila esteja vazia, ou seja, não existem mais *beams* a libertar.

A abordagem adotada permite identificar a existência de ciclos, pois se o número de *beams* presentes no resultado da ordenação topológica for inferior ao número de *beams* identificados como necessários nos passos anteriores, então existe um ciclo e o programa devolve “*Disaster*”. Caso contrário, é devolvida a ordem pela qual os *beams* devem ser removidos, de modo a resolver o problema proposto.

2 Complexidade Temporal

A solução adotada neste projeto foca-se numa otimização espacial para reduzir drasticamente o número de intersecções testadas na construção do grafo de dependências.

2.1 Complexidade Temporal por Método

- **calcBeamBoundaries:** Este método calcula os limites físicos de cada feixe através de cálculos aritméticos diretos e atribuições, sem qualquer recurso a estruturas iterativas.
 - **Complexidade:** $\Theta(1)$
- **createOccupancyGrid:** A inicialização da matriz que mapeia a grelha exige a iteração sobre todas as $R \times C$ células, atribuindo o valor nulo (-1). Seguidamente, o método regista a ocupação de todos os B feixes. Como cada feixe possui uma largura fixa de 1 célula e um comprimento L , a sua colocação executa, no máximo, $O(B \cdot L)$ operações.
 - **Complexidade:** $\Theta(R \times C + B \times L)$
- **traceBeamPath:** Este método simula a trajetória de um feixe para encontrar bloqueios. Tira partido de uma otimização aritmética que salta diretamente para a extremidade do feixe. A partir daí, utiliza um ciclo *while* para avançar pelas células até aos limites da grelha. O percurso máximo possível neste ciclo é ditado pela maior dimensão da matriz, correspondendo a $\max(R, C)$. No melhor caso apenas teremos de verificar uma posição, pelo que o rigor matemático exige utilizar Big-O.
 - **Complexidade:** $O(\max(R, C))$

- **fillAdjacents:** Representa a organização da construção do grafo. Começa por invocar o método *createOccupancyGrid* e, em seguida, itera através de todos os **B** feixes, invocando a função *traceBeamPath* para cada um.
 - **Complexidade:** $O(R \times C + B \times L) + O(B \times \max(R, C))$
 $= O(R \times C + B \times \max(R, C))$
- **getNecessaryBeams:** Consiste num mecanismo de propagação de necessidades suportado por um ciclo *while*. No pior cenário teórico (uma dependência linear em cadeia onde apenas um novo feixe é validado por ciclo), este *while* executa **B** vezes. No seu interior, ocorre a varredura do grafo para verificar os sucessores de cada feixe, o que acarreta um custo de processar vértices e arestas.
 - **Complexidade:** $O(B^2 + B \times E)$
- **topologicalSort:** Implementa o Algoritmo de Kahn, integrando uma fila de prioridade para assegurar o critério de desempate de processar os feixes com o ID mais baixo primeiro. O custo de inserção/remoção na fila de prioridade é logarítmico. Cada um dos **B** vértices e **E** arestas é processado no máximo uma vez.
 - **Complexidade:** $O((B + E) \log B)$

2.2 Composição da Complexidade Temporal Total

A complexidade temporal do algoritmo completo, executado no método *solve()*, resulta da soma das parcelas analisadas acima:

$$T_{total} = \sum Parcelas$$

Ao substituir as funções pelos respetivos limites assintóticos, obtemos:

$$T_{total} = O(B) + O(R \times C + B \times \max(R, C)) + O(B^2 + B \times E) + O((B + E) \log B)$$

Sabe-se pelas restrições físicas do problema que o comprimento de qualquer feixe **L** não pode ser superior à maior dimensão da grelha $\max(R, C)$. Consequentemente, o termo $O(B \times L)$ é dominado e absorvido pelo termo $O(B \times \max(R, C))$. Os termos lineares menores são igualmente omitidos na notação final.

A equação da Complexidade Temporal no Pior Caso:

$$O(R \times C + B \times \max(R, C) + B^2 + B \times E + (B + E) \times \log B)$$

3 Complexidade Espacial

3.1 Análise Espacial por Método

- **solve:** É o centro de comando. Aqui instanciam-se as principais estruturas globais que vão persistir ao longo de todo o algoritmo.
 - Cria o graph (Lista de Adjacências): Aloca espaço para **B** vértices e vai crescer até armazenar **E** arestas.
 - Cria o array inDegree de tamanho **B**.

- Cria o array `isNecessary` de tamanho **B**.
 - Cria a `StringBuilder` final que, no limite, guardará **B** números formatados.
 - **Complexidade:** $\Theta(B + E)$
- **calcBeamBoundaries:** Apenas cria variáveis primitivas temporárias para realizar cálculos. Estas variáveis ocupam um espaço fixo mínimo na pilha de execução e desaparecem imediatamente a seguir.
 - **Complexidade:** $\Theta(1)$
- **createOccupancyGrid:** Executa a instrução para a criação da matriz de ocupações.
 - **Complexidade:** $\Theta(R \times C)$
- **traceBeamPath:** Para evitar que o mesmo feixe seja contabilizado duas vezes como bloqueador, este método cria um conjunto `HashSet<Integer> seenBlockers`. No pior cenário (um feixe gigante que é bloqueado por quase todos os outros feixes no caminho), este conjunto pode chegar a guardar **B** inteiros. Esta memória é libertada a cada nova chamada do método.
 - **Complexidade:** $O(B)$
- **fillAdjacents:** Não aloca estruturas de dados por si mesmo, mas invoca o `createOccupancyGrid` e, de seguida, o `traceBeamPath` **B** vezes. O pico de memória exigido por este bloco é a soma das estruturas ativas simultaneamente: a matriz e o conjunto `HashSet` que é criado e destruído a cada ciclo.
 - **Complexidade:** $O(R \times C + B)$
- **getNecessaryBeams:** Usa apenas variáveis de controlo (como `endColumn`, `count` e booleanos) para navegar pelo grafo e atualizar o vetor `isNecessary` que lhe foi passado por referência. Não cria vetores nem objetos.
 - **Complexidade:** $\Theta(1)$
- **topologicalSort:** Para além de ler o grafo, instância duas estruturas locais:
 - Uma `PriorityQueue` (`pq`) que, se todos os feixes forem necessários e independentes ao mesmo tempo, pode guardar **B** elementos.
 - Uma lista `ArrayList` (`result`) para guardar o output, que também terá o tamanho máximo de **B** elementos.
 - **Complexidade Espacial do Método:** $O(B)$

3.2 Composição Final da Complexidade Espacial

Para obtermos o custo total de espaço do algoritmo, não se soma apenas todos os valores acima cegamente. A memória tem a vantagem de ser reciclável (via *Garbage Collector*). O que interessa é o pico máximo de memória utilizada em simultâneo durante o fluxo do `solve()`.

O momento em que o programa é mais pesado na memória é durante a fase de construção do grafo (`fillAdjacents`). Neste momento exato, o programa tem em memória:

1. Os vetores base de suporte (`inDegree`, `isNecessary`) e o Grafo vazio a encher: $O(B + E)$
2. A Matriz de representação da grelha a funcionar: $O(R \times C)$

3. O *HashSet* do *traceBeamPath* na sua máxima capacidade: $O(B)$

$$S_{total} = O(B + E) + O(R \times C) + O(B)$$

Como o termo $O(B)$ é absorvido por $O(B + E)$, a complexidade espacial global no pior caso simplifica-se para:

$$S_{total} = O(R \times C + B + E)$$

4 Conclusões

A solução apresentada é resultado de várias tentativas de otimização tanto da complexidade temporal, como da espacial.

A abordagem inicial ignorava completamente a utilização de uma matriz, de modo a guardar informação sobre que *beams* habitavam em determinados espaços. Esta abordagem focava-se apenas em analisar o vetor de lista de adjacências de sucessores. Já as intersecções de *beams* eram analisadas com base em lógica matemática, analisando cuidadosamente os valores de linhas e colunas de cada *beam* em relação aos outros.

Esta abordagem foi eventualmente descartada, visto que se o número máximo de *beams* é aproximadamente 10000, então o cálculo das intersecções no pior caso seria $O(B^2)$, o que para este valor máximo é algo como 100 milhões de comparações.

Eventualmente foi reponderado a utilização de uma matriz, visto que as R linhas e C colunas no pior caso originam uma matriz de 200×200 . Como um *beam* em média irá estar situado em uma determinada linha e determinada coluna, para saber se este intercecta outros basta seguir, de acordo com a sua direção, até ao final da linha/coluna que este habita. No pior caso cada *beam* terá de percorrer as R linhas ou C colunas (dependendo da sua direção) subtraindo o seu comprimento. Ainda assim, esta complexidade temporal será aproximadamente $O(R \times B)$ ou $O(C \times B)$, o que tendo em conta as limitações do problema é uma redução considerável da complexidade temporal da secção do problema.

Os pontos negativos não se prendem à solução em questão, mas sim a uma possível incapacidade de uma maior otimização, consequência do conhecimento disponível no momento de implementação.

Os recursos utilizados para elaborar a solução apresentada restringiram-se à utilização do *Google* e do *website GeeksForGeeks* para encontrar documentação necessária, em particular relativamente às complexidades temporais de operações de algumas estruturas de dados como *LinkedList*, *ArrayList*, *ArrayDeque* e *PriorityQueue*, entre outros.